

Problem Set #2 (125 points)

Due date: September 20, 2024, 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start...

- You are not allowed to use material that has not been covered in class.
- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the visible and post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Revise the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: `PS2.py`. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set 2 channel in Microsoft Teams.

*** For all functions below, you may only use sequence statements (variables, constants, expressions, and assignment/return statements), data types covered so far (int, float, string, bool) and conditional statements in your solution. ***

Part 1: Mathematical Expressions

Sometimes, mathematical expressions can be defined using a conditional. Below is an example of one such expression, `example`, which takes in two parameters, `x` and `y`, and returns `True` when the product of `x` and `y` is even and `False` otherwise.

$$\text{example}(x, y) = \begin{cases} \text{True}, & \text{if } x \times y \text{ is even} \\ \text{False}, & \text{otherwise} \end{cases}$$

Convert the following mathematical expressions into functions. Use the name indicated on the left-hand side as the name of your function. All functions must return the output as indicated. For this part, there is no need to provide a docstring, but you must provide annotations in the function definition for input and output.

Function #1 (5 pts)

The parameter `num` is an integer.

$$\text{is_odd}(\text{num}) = \begin{cases} \text{True}, & \text{if } \text{num} \text{ is odd} \\ \text{False}, & \text{otherwise} \end{cases}$$

Function #2 (5 pts)

The parameter `temp` is a float and `to_f` is a bool. When `to_f` is `True`, the temperature is in Celsius and should be converted to Fahrenheit. When it is `False`, the temperature is in Fahrenheit and should be converted to Celsius.

$$\text{temperature_conversion}(\text{temp}, \text{to_f}) = \begin{cases} \frac{9 \times \text{temp}}{5} + 32, & \text{if } \text{to_f} \text{ is True} \\ \frac{(\text{temp} - 32) \times 5}{9}, & \text{otherwise} \end{cases}$$

Function #3 (5 pts)

Chained conditionals can be represented with multiple rows in the expression. Assume all parameters are integers.

$$\text{cascade}(x, y, z) = \begin{cases} x \times y, & \text{if } z \text{ is evenly divisible by } 7 \\ \frac{y - 2z}{x}, & \text{if } x \leq (y - z) \text{ and } x \neq 0 \\ x - z, & \text{otherwise} \end{cases}$$

Part 2: Only Conditionals

Problem 2.1 (10 pts) A bagel shop charges 85 cents per bagel for orders of less than a half-dozen bagels and 55 cents per bagel for orders of a half-dozen or more. Define and implement a function named exactly `get_bill` that takes a positive integer that represents the number of bagels ordered. The function returns the total cost of the orders as a float, and returns -1 if the input is invalid (not an integer and not a positive number).

Examples:

```
>>> get_bill(12)
6.6000000000000005
>>> get_bill(8)
4.4
>>> get_bill(2)
1.7
>>> get_bill(-7)
-1
>>> get_bill(78.0)
-1
```

Problem 2.2 (10 pts) According to our syllabus, we will assign letter grade based on your percentage grade at the end of the semester as the following table.

A	>=93.0
A-	>=90.0
B+	>=87.0
B	>=83.0
B-	>=80.0
C+	>=77.0
C	>=70.0
D	>=60.0
F	<60.0

Define and implement a function named exactly `get_letter_grade` that given a floating point number, it returns a string with the corresponding letter grade. Your function must use `if/elif/else` structure.

Examples:

```
>>> get_letter_grade(100)
'A'
>>> get_letter_grade(92.99)
'A-'
>>> get_letter_grade(81.5)
'B-'
>>> get_letter_grade(73.8)
'C'
>>> get_letter_grade(3)
'F'
```

Problem 2.3 (10 pts) Define and implement a function named exactly `intersect`. Your function should take six numerical arguments, $x_1, y_1, radius_1, x_2, y_2$ and $radius_2$, that describe the circle centered at (x_1, y_1) with radius $radius_1$, and the circle centered at (x_2, y_2) with radius $radius_2$. The function returns `True` if the two circles intersect and `False` otherwise. Intersecting circles must touch or overlap at one or more points. There are a few scenarios for circle intersections:

- One circle is completely inside the other: The centers of the circles coincide, and the distance between the centers is less than the difference between the radii.
- Circles have the same center: The distance between the centers is zero, which is less than the sum of the radii
- Circles have distinct centers: The distance between the centers is less than the sum of the radii (the circles touch or overlap at one or more points)
- Circles are tangent: The distance between the centers is equal to the sum of the radii (the circles touch at a single point)

As a reference, the distance between two points can be calculated using the following formula:

$$distance = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

⊖ To receive any credit for this problem, you are not allowed to import and use the `math` module!

Examples:

```
>>> intersect(2, 3, 12, 15, 28, 10)
False
>>> intersect(3, 4, 5, 14, 18, 8)
False
>>> intersect(1.0, 1, 3, 5, 4.0, 2)
True
```

Problem 2.4 (10 pts) Define and implement a function named exactly `get_num`. Your function should take three numerical arguments, `num, bound_1` and `bound_2`, where `bound_1` is not necessarily less than `bound_2`. If `num` is between the two bounds, just return `num`, otherwise, if `num` is less than the lower bound, return the lower bound, or if `num` is greater than the upper bound, return the upper bound.

⊖ To receive any credit for this problem, you cannot use the built-in functions `min()` or `max()`. However, you are allowed define your own `get_min()` or `get_max()` functions to simplify the logics of your function.

Examples:

```
>>> get_num(1,3,5)
3
>>> get_num(4.7,3,5)
4.7
>>> get_num(6,5,3)
5
```

Problem 2.5 (10 pts) Define and implement a function named exactly `how_many`. Your function should take one argument that represents a number of eggs. The function returns the smallest integer number of cartons required to hold the provided quantity of eggs, where a carton may hold up to 12 eggs.

🚫 To receive any credit for this problem, you are not allowed to import and use the `math` module!

Examples:

```
>>> how_many(5)
1
>>> how_many(24)
2
>>> how_many(6548)
546
```

Problem 2.6 (10 pts) Define and implement a function named exactly `is_even_positive`. Your function should take one argument. The function returns `True` if the argument is an integer, and it is positive, and it is even (all 3 must be `True`), or `False` otherwise. Your function should not raise exceptions if the input is not an integer.

Examples:

```
>>> is_even_positive(12)
True
>>> is_even_positive(12.0)
False
>>> is_even_positive("hello")
False
>>> is_even_positive(33)
False
>>> is_even_positive(-12)
False
```

Problem 2.7 (10 pts) Define and implement a function named exactly `is_triangle`. Your function should take three numerical arguments that represent the lengths of the sides of a triangle. The function returns `True` if such a triangle exists and `False` otherwise. All sides of a legal triangle must be positive, return `False` when this condition is not met.



Hint:

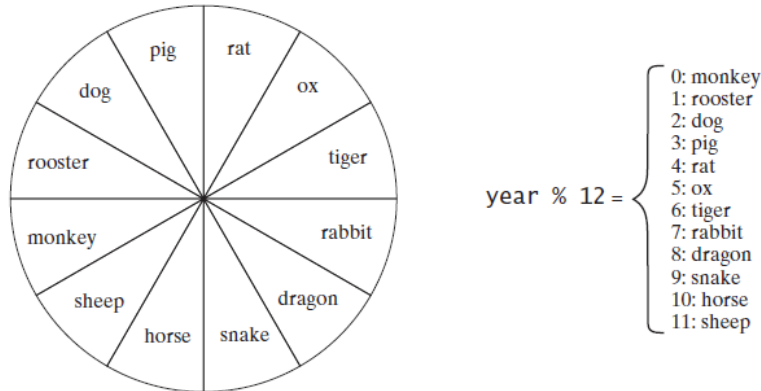
The triangle inequality states that for any triangle, the sum of the lengths of any two sides must be greater than the length of the remaining side.

Examples:

```
>>> is_triangle(3, 4, 5)
True
>>> is_triangle(3, -4, 5)
False
>>> is_triangle(2.5, 3.75, 5.1)
True
```

Problem 2.8 (10 pts) The Chinese Zodiac sign is based on a 12-year cycle, and each year in this cycle is represented by an animal. For this problem, we will use the chart shown below. Define and implement a function named exactly `get_chinese_zodiac`. Your function should take one argument, `year`. The function returns a string (all lowercase) with the name of the animal that represents that year. Use an *if/elif/else* structure.

⊖ To receive any credit for this problem, you are not allowed to use data types that have not been covered yet in this course!



Examples:

```
>>> get_chinese_zodiac(2003)
'sheep'
>>> get_chinese_zodiac(2022)
'tiger'
>>> get_chinese_zodiac(2023)
'rabbit'
>>> get_chinese_zodiac(2020)
'rat'
>>> get_chinese_zodiac(2021)
'ox'
>>> get_chinese_zodiac(2031)
'pig'
```

Part 3: Who is the best?

People often use terms like “best” to describe how to solve a particular problem, but “best” could be relative and sometimes isn’t based on anything; it’s simply the “best” because everyone says it is! In this problem, we will define “best” number as the middle-most of five unique integers. Why is this the best? It is not too high or too low; it’s “just right”.

Before you begin, take the time to design your approach. Trying to find the middle of a bunch of numbers can be dreadful, especially if you start writing conditionals without a plan! To narrow our focus, we will ask, “*How can I find the 3rd largest number?*” that way, we can find the maximum, then the second maximum, and finally the third maximum, which is our “just-right” number!

⊖ To receive any credit for this problem, you cannot use the built-in functions `min()` or `max()`.

Problem 3.1 (10 pts) Define and implement a function named exactly `get_maximum` that takes two integers and returns whichever is larger. Your solution must use one single conditional.

Examples:

```
>>> get_maximum(5, 7)
7
>>> get_maximum(4, 2)
4
```

Problem 3.2 (10 pts) Next, you need to find the maximum again, but with some extra information: the previous maximum you found! Define and implement a function named exactly `get_maximum_below` that takes three integers as arguments, the two integers we should compare to find the maximum of, and the third as a limit, the previous maximum you found, and returns an integer according to the following logic:

- If the first integer is greater or equal to the limit, return the second integer.
- If the second integer is greater or equal to the limit, return the first integer.
- If it's neither of those, call `get_maximum` with the first two integers and return the value you got back.

Examples:

```
>>> get_maximum_below(4, 2, 12)
4
>>> get_maximum_below(4, 2, 1)
2
```

Problem 3.3 (10 pts) Once you have implemented and tested the previous functions, it is time to calculate the “best” integer. Define and implement a function named exactly `get_the_best`, which will take five unique integers and return the middle-most integer using the two previous functions you have designed. 🚫 You cannot use any other functions.

Examples:

```
>>> get_the_best(6, 8, 10, 2, 4)
6
>>> get_the_best(32, 83, 772, 1000, 1983)
772
```

💡 Hint:

We should always look for ways to make our lives easier when coding. This problem forces us to do just that by splitting this task into three less complex steps. Take each step in isolation, and avoid getting caught up in how the parts will work together; only try to solve the problem in front of you.

The first two functions only compare two numbers together, but we'll need to compare five. How can we use those functions to do that? First, consider how to use `get_maximum` to get the maximum of three numbers. We'd need to compare two numbers, which returns the maximum between the first two. If we call the function again using that returned value along with our third input, this would give us the maximum of the three values. Following this pattern, how can you extend this to four values? Then to five?

Once you find the maximum value of the five inputs, use that maximum to find the maximum value below that and then use that value to find the maximum value below that second maximum.